FSStat Library

Assignment #02 — v1.0.0-20260412

Concurrent and Distributed Programming

]Eduard Toni Alexandru

Approaches: Event Loop (Vert.x), Reactive (RxJava), Virtual Threads
Language: Java 21
Libraries: Vert.x 4, RxJava 3

September 20, 2026

0 Contents

1 Analysis **2**
1.1 Problem Description . . . 2
1.2 Optional Extension . . . 2
1.3 Key Challenges . . . 2

2 Design **3**
2.1 Architectural Overview . . . 3
2.2 Push vs. Pull Update Model . . . 3
2.3 Core Interfaces and Data Model . . . 3
2.3.1 FSStatLib . . . 3
2.3.2 FSUpdateListener . . . 4
2.3.3 FSReport and FSStats . . . 4
2.4 View-Side Polling . . . 4
2.5 UML Class Diagram . . . 6

3 Implementation **6**
3.1 Shared Foundations . . . 6
3.2 Event Loop — FSStatLibVertx . . . 7
3.2.1 Approach . . . 7
3.2.2 Completion Tracking . . . 7
3.2.3 Update Strategy . . . 7
3.2.4 Stopping . . . 7
3.2.5 Design Notes . . . 7
3.3 Reactive Programming — FSStatLibReactive . . . 8
3.3.1 Approach . . . 8
3.3.2 Directory Scan as a Flowable . . . 8
3.3.3 Pipeline . . . 8
3.3.4 Stopping . . . 9
3.3.5 Concurrency . . . 9
3.3.6 Why Not MapReduce . . . 9
3.4 Virtual Threads — FSStatLibVT . . . 9
3.4.1 Approach . . . 9
3.4.2 MapReduce per Directory . . . 9
3.4.3 Update Strategy . . . 10
3.4.4 Stopping . . . 10
3.4.5 Why MapReduce Fits Here and Not Elsewhere . . . 11

4 Conclusion **11**

1 Analysis

1.1 Problem Description

The goal of this assignment is to design and implement a library called `FSStatLib` that asynchronously computes statistics about the sizes of files contained in a given directory `D`, including all of its subdirectories recursively.

The library exposes two methods:

```
void getFSReport(Path dir, long maxFS, int nb, FSUpdateListener
    listener);
FSReport getCurrentReport();
void stop();
```

`getFSReport` launches an asynchronous scan and must compute:

- The **total number of files** found recursively under `dir`.
- The **distribution of file sizes**: given a maximum file size `MaxFS` and a number of bands `NB`, the range $[0, \texttt{MaxFS}]$ is divided into `NB` equal intervals. Files larger than `MaxFS` are counted in a dedicated overflow bucket, giving `NB + 1` size ranges in total.

`getCurrentReport` allows any caller to query the current state of the scan at any moment, returning an immutable snapshot of the statistics accumulated so far.

1.2 Optional Extension

The assignment also requires an interactive extension providing:

- The ability to **stop** report generation at any time.
- **Dynamic, incremental visualisation** of statistics while the scan is running.

A minimal Swing GUI demonstrates these features, with start/stop controls and a histogram panel that updates in real time.

1.3 Key Challenges

Recursive directory traversal. A directory tree of arbitrary depth and width must be explored, requiring recursive logic that correctly signals completion only when the entire tree has been scanned.

Asynchronous coordination. Each I/O operation is asynchronous or concurrent. Tracking when all operations across an unbounded tree have finished requires careful design.

UI responsiveness. Intermediate results must be pushed to the UI at a controlled rate to avoid overwhelming the Event Dispatch Thread (EDT).

Separation of concerns. The library should be unaware of UI timing. The decision of how often to refresh the view belongs to the view layer, not the library.

Cancellability. The scan must be interruptible at any point without leaving threads or resources in an inconsistent state.

Programming discipline. Each of the three implementations must adhere strictly to its paradigm without mixing styles.

2 Design

2.1 Architectural Overview

The application follows an **MVC (Model-View-Controller)** architecture, separating UI concerns from library logic. The library is hidden behind the `FSStatLib` interface, allowing the three implementations to be swapped transparently.

A key architectural decision is the adoption of a **pull-based update model**: the library does not call any update callback during the scan. Instead, the View queries the Model at fixed time intervals using a Swing `Timer`. This means:

- The library has **no concept of time** and no throttling logic.
- The UI refresh rate is controlled **entirely by the View**.
- `FSUpdateListener` is reduced to a single `onComplete` callback.

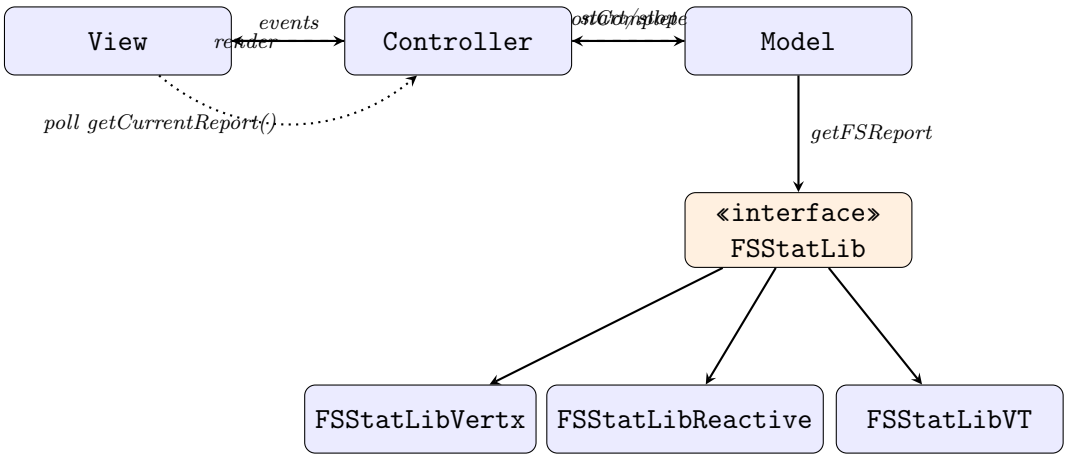


Figure 1: MVC architecture with pull-based updates

2.2 Push vs. Pull Update Model

The pull-based model is superior to a push-based model for this use case:

Aspect	Push (discarded)	Pull (adopted)
Who controls timing	Library	View
Library knows about time	Yes (throttle)	No
UI refresh rate	Library-driven	View-driven
Library complexity	Higher	Lower
Separation of concerns	Weaker	Stronger

2.3 Core Interfaces and Data Model

2.3.1 FSStatLib

```
public interface FSStatLib {
    void getFSReport(Path dir, long maxFS, int nb, FSUpdateListener
   listener);
    FSReport getCurrentReport();
    void stop();
}
```

2.3.2 FSUpdateListener

Reduced to a single method — the library only signals completion:

```
public interface FSUpdateListener {
    void onComplete(FSReport report);
}
```

2.3.3 FSReport and FSStats

`FSReport` is an immutable snapshot implemented as a Java record:

```
public record FSReport(long totalFiles, long[] bands) {}
```

`FSStats` is a mutable, thread-safe accumulator with `NB + 1` bands (index `NB` is the overflow bucket). It exposes two key methods:

```
public synchronized void addFile(int band) {
    this.totalFiles += 1;
    this.bands[band] += 1;
}

// Used by the MapReduce merge step in the VT implementation
public synchronized void mergeReport(FSReport report) {
    this.totalFiles += report.totalFiles();
    for (int i = 0; i < report.bands().length; i++) {
        this.bands[i] += report.bands()[i];
    }
}

public synchronized FSReport snapshot() {
    return new FSReport(totalFiles, bands.clone());
}
```

The band index is computed as:

```
int band = Math.min((int)(fileSize / bandSize), nb); // nb =
   overflow bucket
```

2.4 View-Side Polling

The View owns a Swing `Timer` that polls the Controller every 200 ms:

```
private Timer refreshTimer;

public void startPolling() {
    refreshTimer = new Timer(200, e ->
        render(controller.getCurrentReport()));
```

```
    refreshTimer.start();
}

public void stopPolling() {
    if (refreshTimer != null) refreshTimer.stop();
}
```

The Controller wires start, stop, and completion:

```
public void start(String folder, int nb, long maxSize) {
    view.startPolling();
    model.setListener(new ModelListener() {
        public void onComplete(FSReport report) {
            view.stopPolling();
            SwingUtilities.invokeLater(() -> view.render(report));
        }
    });
    model.start(Path.of(folder), maxSize, nb);
}
```

2.5 UML Class Diagram

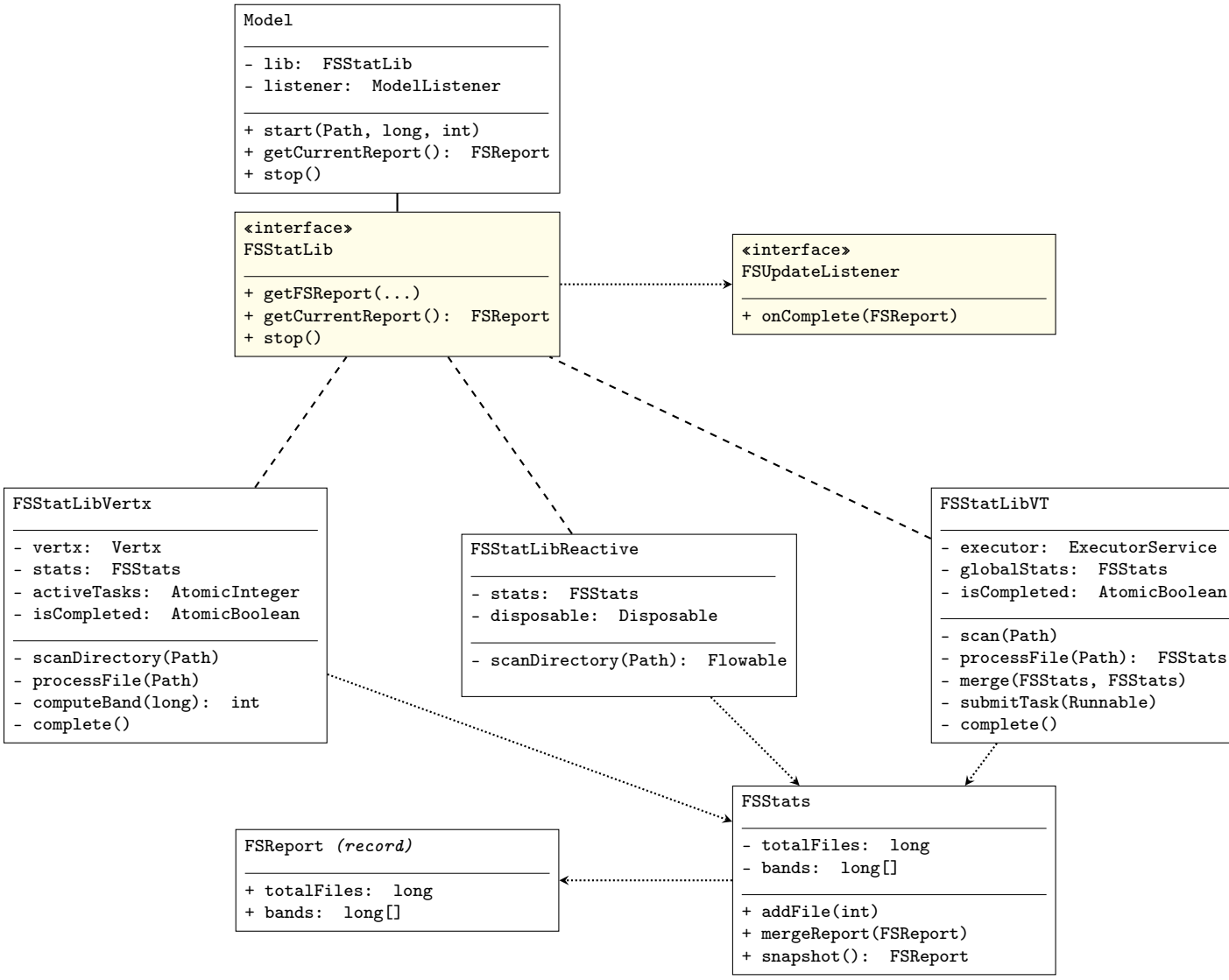


Figure 2: Full UML class diagram

3 Implementation

3.1 Shared Foundations

All three implementations share the same data model (`FSStats`, `FSReport`), the same interface (`FSStatLib`), and the same band computation:

```
int band = Math.min((int)(fileSize / bandSize), nb);
```

Each returns the current snapshot on demand:

```
public FSReport getCurrentReport() {
    return stats != null ? stats.snapshot() : new FSReport(0, new
   long[nb + 1]);
}
```

What differs is entirely how I/O concurrency, recursive coordination, and completion detection are handled — each according to its paradigm.

3.2 Event Loop — FSStatLibVertx

3.2.1 Approach

The Vert.x implementation follows a **callback-based, event-loop** model. All filesystem operations are non-blocking: instead of waiting for a result, a callback is registered and the Vert.x event loop invokes it when the OS responds. No thread is ever blocked.

3.2.2 Completion Tracking

The fundamental challenge is knowing when the entire recursive async chain has finished. An `AtomicInteger activeTasks` is used as a counter: incremented before every async call, decremented in a `finally` block inside every callback. When it reaches zero, `complete()` is called exactly once, guarded by an `AtomicBoolean`:

```
fs.props(entry).onComplete(propRes -> {
    try {
        if (propRes.result().isDirectory()) {
            scanDirectory(Path.of(entry));
        } else {
            processFile(Path.of(entry));
        }
    } finally {
        if (activeTasks.decrementAndGet() == 0) {
            complete();
        }
    }
});
```

3.2.3 Update Strategy

Following the pull-based architecture, the library performs no `onUpdate` calls. It only calls `listener.onComplete()` when the scan finishes. The View polls `getCurrentReport()` via its own timer.

3.2.4 Stopping

Setting `isCompleted` to `true` causes all in-flight callbacks to exit early. `complete()` is also guarded by `getAndSet` so it fires at most once.

3.2.5 Design Notes

- A single `Vertx` instance is created once as a class field, not per call, to avoid the overhead of spinning up a new event loop on every invocation.
- `processFile` performs a dedicated `fs.props()` call to retrieve the file size, keeping the async discipline consistent throughout.
- MapReduce is **not applicable** here: Vert.x is built around a single-threaded event loop where blocking (e.g. waiting on futures to merge) is forbidden. The natural coordination pattern is the shared accumulator with an async counter.

3.3 Reactive Programming — FSStatLibReactive

3.3.1 Approach

The RxJava implementation treats the directory traversal as a **stream of paths** (`Flowable<Path>`). The entire scan is expressed as a declarative pipeline of operators. RxJava manages scheduling, backpressure, and lifecycle automatically — no manual counters or flags are needed.

3.3.2 Directory Scan as a Flowable

```
private Flowable<Path> scanDirectory(Path dir) {
    return Single.fromCallable(() -> {
                try (var stream = Files.list(dir)) {
                    return stream.toList();
                }
            })
            .subscribeOn(Schedulers.io())
            .flatMapPublisher(entries -> Flowable.fromIterable(
    entries)
                    .flatMap(entry -> Single.fromCallable(() ->
    entry)
                            .subscribeOn(Schedulers.io())
                            .flatMapPublisher(path -> {
                                if (Files.isDirectory(path)) {
                                    return scanDirectory(path);
                                } else {
                                    return Flowable.just(path);
                                }
                            }), DEFAULT_CONCURRENCY)
            );
}
```

The construction works in layers:

1. `Single.fromCallable` wraps the blocking `Files.list()` call.
2. `.subscribeOn(Schedulers.io())` offloads it to a dedicated I/O thread pool.
3. `Flowable.fromIterable` emits entries one at a time, *lazily*.
4. `flatMap(..., 8)` processes up to 8 entries concurrently, each shifted onto an I/O thread via a nested `Single.fromCallable + subscribeOn`.
5. For directories, `scanDirectory` recurses, returning a sub-stream whose completion propagates naturally through `flatMap`.
6. For files, `Flowable.just(path)` emits one item and completes immediately.

`flatMap` completes only when *all* inner flowables complete, so the top-level stream terminates correctly when the entire tree is scanned — with no manual tracking.

3.3.3 Pipeline

```
disposable = scanDirectory(dir)
    .map(Files::size)
    .map(size -> Math.min((int)(size / bandSize), nb))
    .doOnNext(stats::addFile)
```

```
    .doOnComplete(() -> listener.onComplete(stats.snapshot()))
    .subscribe(x -> {}, err -> listener.onComplete(stats.snapshot())
  );
```

Each operator has a single responsibility:

Operator	Purpose
`.map(Files::size)`	Retrieve file size (on `Schedulers.io()`)
`.map(...computeBand)`	Map size to band index
`.doOnNext(stats::addFile)`	Accumulate into shared `FSStats`
`.doOnComplete(...)`	Notify listener on completion

No buffering or time-based operators appear in the pipeline — update timing is entirely the View's responsibility.

3.3.4 Stopping

`disposable.dispose()` cancels the entire stream, propagating upstream through every operator automatically.

3.3.5 Concurrency

`DEFAULT_CONCURRENCY = 8` limits simultaneous I/O operations. Filesystem scanning is I/O-bound, not CPU-bound: threads spend most of their time waiting for disk responses. The appropriate concurrency limit is therefore not the number of CPU cores but a value tuned to the storage medium (8–16 for SSDs, 1–4 for HDDs).

3.3.6 Why Not MapReduce

MapReduce is **not applicable** in RxJava because the stream already implicitly encodes it: the `map` phase is the per-file transformation, and the `doOnNext` accumulation is semantically equivalent to a reduce. Applying an explicit MapReduce pattern would be a redundant re-expression of what the operators already provide.

3.4 Virtual Threads — FSStatLibVT

3.4.1 Approach

The Virtual Threads implementation uses **structured concurrency with Java 21 virtual threads**. Blocking I/O calls are written in a straightforward imperative style — the JVM automatically parks and unmounts the virtual thread while it waits for I/O, without blocking a real OS thread. This makes blocking code effectively non-blocking at the platform level.

3.4.2 MapReduce per Directory

Unlike the other two implementations, the Virtual Threads version adopts a **MapReduce pattern scoped to each directory level**:

- **Map** — for each entry in a directory, a virtual thread is spawned. Files return a partial `FSStats`; subdirectories recurse.

- **Reduce** — once all futures for a directory complete, their partial results are merged into a single `FSStats` for that directory level, then merged into the global accumulator.

This pattern is natural here because virtual threads *can* block on `Future.get()` cheaply — the JVM parks the virtual thread rather than wasting a real OS thread.

```
private void scan(Path dir) throws Exception {
    if (isCompleted.get()) return;

    List<Future<FSStats>> futures;

    // MAP: spawn one virtual thread per entry
    try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir
   )) {
        futures = StreamSupport.stream(stream.spliterator(), false)
                .map(path -> executor.submit(() -> {
                    if (isCompleted.get()) return new FSStats(nb);
                    if (Files.isDirectory(path)) {
                        scan(path);            // recurse; subdir
   merges itself
                        return new FSStats(nb);
                    } else {
                        return processFile(path);
                    }
                }))
                .toList();
    }

    // REDUCE: merge partial results for this directory
    FSStats dirStats = new FSStats(nb);
    for (Future<FSStats> future : futures) {
        merge(dirStats, future.get()); // parks virtual thread
   cheaply
    }

    merge(globalStats, dirStats);
}
```

`merge` uses the efficient `mergeReport` method on `FSStats`, which operates in $O(NB)$ time rather than $O(files)$:

```
private void merge(FSStats dst, FSStats src) {
    dst.mergeReport(src.snapshot());
}
```

3.4.3 Update Strategy

No `onUpdate` calls appear anywhere in the implementation. The View polls `getCurrentReport()` on its own schedule.

3.4.4 Stopping

`executor.shutdownNow()` interrupts all running virtual threads. The `isCompleted` flag causes tasks that started before shutdown to exit early via guard checks.

3.4.5 Why MapReduce Fits Here and Not Elsewhere

Virtual threads are the only paradigm where `Future.get()` is cheap. In Vert.x, blocking the event loop is forbidden. In RxJava, the stream operators already encode the pattern implicitly. The comparison is summarised below:

Aspect	**Vert.x**	**RxJava**	**Virtual Threads**
I/O model	Non-blocking (event loop)	Non-blocking (schedulers)	Blocking (parked by JVM)
Code style	Callback-based	Declarative pipeline	Imperative/sequential
Coordination	AtomicInteger counter	flatMap completion	Future.get() + merge
MapReduce	Not applicable	Implicit in operators	Explicit per directory
Accumulation	Shared FSStats	Shared FSStats	Partial + merge

4 Conclusion

All three implementations satisfy the full assignment requirements: recursive directory scanning, `NB + 1` size bands with overflow tracking, dynamic incremental updates, and cancellability.

The adoption of a **pull-based update model** is the most significant architectural decision: by removing `onUpdate` from the library interface and delegating refresh timing to the View's `Timer`, the libraries become simpler, time-agnostic, and better separated from UI concerns. The update rate can be changed in a single place without touching any library implementation.

Each implementation strictly adheres to its paradigm:

- **Vert.x** demonstrates the event-loop model: non-blocking callbacks with explicit async coordination via an atomic task counter.
- **RxJava** demonstrates reactive programming: a declarative stream pipeline where operators handle scheduling, concurrency, backpressure, and lifecycle automatically.
- **Virtual Threads** demonstrates structured concurrency: imperative, readable code with a per-directory MapReduce pattern, made efficient by the JVM's ability to park virtual threads cheaply on blocking calls.

The common `FSStatLib` interface, shared `FSStats` accumulator, and MVC architecture ensure that all three implementations are interchangeable without any change to the application layer.